Studio Nx

JS 300 — 내보내기 창

2026-10-20

JavaScript 300 — 내보내기 창 (`sequencer-export.js`, `studioNx.html`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-export.js`, `studionx/frontend/static/js/pages/sequencer-audio.js` (`exportSnapshot`와 선택 표시), `studionx/frontend/studioNx.html` (`#export-modal`), `studionx/frontend/static/css/pages/studio.css` (`.ex-*`)
작업	S1-11b WAV 출력: 내보내기 창과 다운로드 (FR-301, FR-306, FR-307)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 299 오프라인 렌더링, 다음 없음 (S1-11b의 마지막)
화면 목업	`docs/page_mockups/export_modal/` (6장)

1. 하는 일

FILE 메뉴의 Export..., 상단의 ↓ 버튼, 단축키 Ctrl+Shift+R이 여는 창입니다. 설정, 진행 중, 완료 세 화면으로 바뀝니다. 소리 계산은 `audio/` 폴더(297, 298, 299)가 하고, 이 파일은 화면의 값을 읽고 진행률을 보여 주고 파일을 내려받게 하는 일만 합니다.

```
창 열기(openExport)
   화면 스냅샷(exportSnapshot) + 지난번 선택 + 기본값
        ▼
설정 화면 ── 값이 바뀔 때마다 refreshForm (범위 설명, 요약 줄, 내보내기 버튼 켜고 끄기)
        │  [Export]
        ▼
진행 화면 ── renderOffline(299) ─▶ 진행률 막대, 남은 시간 어림
        │       [Stop] 또는 Esc ─▶ 설정 화면으로 돌아감 (파일 없음)
        ▼
인코딩: 피크 ─▶ 정규화 배율 ─▶ encodeWav(297)
        ▼
완료 화면 ── 파일명, 길이, 크기, 피크, 걸린 시간 ── [Download] ─▶ 브라우저 다운로드
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer.js`의 `initExport()` 호출	명령 `file.export`를 등록합니다. 메뉴, 커맨드 팔레트, 단축키가 같은 명령을 부릅니다.
`sequencer-ui.js`의 ↓ 버튼	이전의 "Phase 3에서 구현" 안내를 `runCommand('file.export')`로 바꿨습니다.
`sequencer.js`의 `initShortcuts`	Ctrl+Shift+R을 브라우저의 강력 새로고침 대신 이 창으로 연결합니다.
`sequencer.js`의 `applyProjectInfo`	파일명에 쓸 프로젝트 이름을 `document.body.dataset.projectTitle`에 둡니다.

3. 코드

3.1 화면의 값 모으기 (`sequencer-audio.js`)

```
export function exportSnapshot() {
  const loop = getLoop();
  return {
    tracks: playbackTracks(),
    audioClips: collectAudioClips(),
    midiClips: collectMidiClips(),
    getInstruments: playbackInstruments,
    getEffects: playbackEffects,
    bpm: currentBpm(),
    tempoChanges: [],
    loop: loop.on && loop.length > 0
      ? { start: Math.round(loop.start * TICKS_PER_BAR), end: Math.round((loop.start + loop.length) * TI
CKS_PER_BAR) }
      : null,
    loadSampleBytes: loadSampleBytes,
  };
}
```

클립 정보에 `selected` 값이 추가되었습니다. 화면의 클립 요소에 `is-selected` 클래스가 있는지를 담습니다.

3.2 창의 값 읽기와 계산

```
function readOptions() {
  const checked = document.querySelector('input[name="ex-range"]:checked');
  return {
    mode: checked ? checked.value : 'all',
    fromBar: Number(document.getElementById('ex-from').value),
    toBar: Number(document.getElementById('ex-to').value),
    sampleRate: segmentValue('ex-rate', 'rate') || 44100,
    bitDepth: segmentValue('ex-depth', 'depth') || 16,
    normalize: document.getElementById('ex-norm').checked,
    normalizeDb: clampNumber(document.getElementById('ex-norm-db').value, -12, 0, -0.3),
    tailSec: clampNumber(document.getElementById('ex-tail').value, 0, MAX_TAIL_SECONDS, 0),
    fileName: sanitizeFileName(document.getElementById('ex-file').value.replace(/\.wav$/i, '')),
  };
}
```

```
function rangeFor(o) {
  return resolveExportRange({
    mode: o.mode, clips: allClips(), loop: snap.loop, selected: selectedClips(),
    fromBar: o.fromBar, toBar: o.toBar, ticksPerBar: TICKS_PER_BAR,
  });
}
```

```
function describeRange(range) {
  const a = Math.floor(range.startTick / TICKS_PER_BAR) + 1;
  const b = Math.max(a, Math.ceil(range.endTick / TICKS_PER_BAR));
  return 'bar ' + a + ' – ' + b + ' · ' + formatDuration(rangeSeconds(range.startTick, range.endTick, sn
```

```
ap.bpm, snap.tempoChanges));
}
```

```
function computeAutoTail() {
  const effects = [];
  const layers = [];
  snap.tracks.forEach(function (t) {
    (snap.getEffects(t.track_id) || []).forEach(function (e) { effects.push(e); });
    (snap.getInstruments(t.track_id) || []).forEach(function (l) { layers.push(l); });
  });
  return autoTailSeconds(effects, layers, snap.bpm);
}
```

3.3 설정 화면 갱신

```
function refreshForm() {
  if (!snap) return;
  const o = readOptions();

  // 범위별 설명과 사용 가능 여부
  const all = rangeFor({ mode: 'all', fromBar: o.fromBar, toBar: o.toBar });
  document.getElementById('ex-len-all').textContent = all.ok ? describeRange(all) : 'Nothing to export y
et';
  const loop = rangeFor({ mode: 'loop', fromBar: o.fromBar, toBar: o.toBar });
  document.getElementById('ex-len-loop').textContent = loop.ok ? describeRange(loop) : 'No loop region';
  const sel = rangeFor({ mode: 'selected', fromBar: o.fromBar, toBar: o.toBar });
  const selCount = selectedClips().length;
  document.getElementById('ex-len-selected').textContent = sel.ok ? selCount + (selCount === 1 ? ' clip
 · ' : ' clips · ') + describeRange(sel) : 'Select clips on the timeline first';
  const custom = rangeFor({ mode: 'custom', fromBar: o.fromBar, toBar: o.toBar });
  document.getElementById('ex-len-custom').textContent = custom.ok ? describeRange(custom) : '—';

  const loopRadio = document.querySelector('input[name="ex-range"][value="loop"]');
  const selRadio = document.querySelector('input[name="ex-range"][value="selected"]');
  loopRadio.disabled = !loop.ok;
  selRadio.disabled = !sel.ok;
  document.getElementById('ex-opt-loop').classList.toggle('is-off', !loop.ok);
  document.getElementById('ex-opt-selected').classList.toggle('is-off', !sel.ok);
  if ((o.mode === 'loop' && !loop.ok) || (o.mode === 'selected' && !sel.ok)) {
    document.querySelector('input[name="ex-range"][value="all"]').checked = true;
    o.mode = 'all';
  }
  document.getElementById('ex-from').disabled = o.mode !== 'custom';
  document.getElementById('ex-to').disabled = o.mode !== 'custom';
  document.getElementById('ex-custom').classList.toggle('is-off', o.mode !== 'custom');
  document.getElementById('ex-selected-note').hidden = o.mode !== 'selected';

  // 비트 깊이 안내와 정규화 칸
  document.getElementById('ex-depth-note').textContent = o.bitDepth === 16
    ? '16-bit adds a little dither noise so quiet tails stay smooth.'
    : (o.bitDepth === 24 ? '24-bit keeps more detail for further mixing.' : '32-bit float cannot clip in
side the file and is best for further editing.');
  document.getElementById('ex-norm-db').disabled = !o.normalize;

  // 요약 줄과 내보내기 버튼
```

```
  const range = rangeFor(o);
  const msg = document.getElementById('ex-msg');
  if (range.ok) {
    const seconds = rangeSeconds(range.startTick, range.endTick, snap.bpm, snap.tempoChanges) + o.tailSe
c;
    const frames = Math.ceil(seconds * o.sampleRate);
    document.getElementById('ex-sum-len').textContent = formatDuration(seconds);
    document.getElementById('ex-sum-fmt').textContent = 'WAV · ' + (o.sampleRate / 1000) + ' kHz · ' +
(o.bitDepth === 32 ? '32-bit float' : o.bitDepth + '-bit') + ' stereo';
    document.getElementById('ex-sum-size').textContent = '≈ ' + formatBytes(wavByteLength(frames, 2, o.b
itDepth));
    msg.hidden = true;
  } else {
    document.getElementById('ex-sum-len').textContent = '—';
    document.getElementById('ex-sum-fmt').textContent = '—';
    document.getElementById('ex-sum-size').textContent = '—';
    msg.textContent = reasonText(range.reason);
    msg.hidden = false;
  }
  document.getElementById('ex-go').disabled = !range.ok;
}
```

3.4 열기

```
export function openExport() {
  const dlg = document.getElementById('export-modal');
  if (!dlg || dlg.open) return;
  snap = exportSnapshot();
  result = null;
  rendering = false;
  cancelFlag = false;

  const prefs = loadPrefs();
  const projectRate = Number(document.body.dataset.sampleRate) === 48000 ? 48000 : 44100;
  pickSegment('ex-rate', 'rate', prefs.sampleRate === 48000 || prefs.sampleRate === 44100 ? prefs.sample
Rate : projectRate);
  pickSegment('ex-depth', 'depth', [16, 24, 32].indexOf(prefs.bitDepth) >= 0 ? prefs.bitDepth : 16);
  document.getElementById('ex-norm').checked = !!prefs.normalize;
  document.getElementById('ex-norm-db').value = String(clampNumber(prefs.normalizeDb, -12, 0, -0.3));

  document.querySelector('input[name="ex-range"][value="all"]').checked = true;
  const all = rangeFor({ mode: 'all' });
  document.getElementById('ex-from').value = '1';
  document.getElementById('ex-to').value = String(all.ok ? Math.max(1, Math.ceil(all.endTick / TICKS_PER
_BAR)) : 1);
  document.getElementById('ex-tail').value = String(computeAutoTail());
  document.getElementById('ex-file').value = exportFileName(document.body.dataset.projectTitle, new Date
());

  showView('form');
  refreshForm();
  dlg.showModal();
}
```

3.5 내보내기, 중지, 다운로드, 닫기

```
async function startExport() {
  const o = readOptions();
  const range = rangeFor(o);
  if (!range.ok || rendering) return;
  savePrefs(o);
  rendering = true;
  cancelFlag = false;
  startedAt = Date.now();
  document.getElementById('ex-stop').disabled = false;
  document.getElementById('ex-prog-title').textContent = 'Rendering…';
  document.getElementById('ex-prog-note').textContent = 'Keep this window open. You can stop it at any t
ime.';
  setProgress(0);
  showView('progress');

  let audio = snap.audioClips;
  let midi = snap.midiClips;
  if (o.mode === 'selected') {
    audio = audio.filter(function (c) { return c.selected; });
    midi = midi.filter(function (c) { return c.selected; });
  }

  try {
    const out = await renderOffline({
      tracks: snap.tracks, audioClips: audio, midiClips: midi,
      getInstruments: snap.getInstruments, getEffects: snap.getEffects,
      bpm: snap.bpm, tempoChanges: snap.tempoChanges,
      startTick: range.startTick, endTick: range.endTick, tailSec: o.tailSec, sampleRate: o.sampleRate,
      loadSampleBytes: snap.loadSampleBytes,
      onProgress: setProgress,
      isCancelled: function () { return cancelFlag; },
    });
    document.getElementById('ex-prog-title').textContent = 'Writing the file…';
    await nextFrame();
    const peak = peakOfChannels(out.channels);
    const gain = o.normalize ? normalizeGain(peak, o.normalizeDb) : 1;
    const bytes = encodeWav(out.channels, out.sampleRate, o.bitDepth, { gain: gain, dither: true });
    result = { bytes: bytes, fileName: o.fileName };

    document.getElementById('ex-done-name').textContent = o.fileName + '.wav';
    document.getElementById('ex-done-meta').textContent = formatDuration(out.frames / out.sampleRate) +
' · ' + formatBytes(bytes.length);
    const seconds = Math.max(1, Math.round((Date.now() - startedAt) / 1000));
    if (peak > 0) {
      const peakDb = 20 * Math.log10(peak * gain);
      document.getElementById('ex-done-note').textContent = 'Peak ' + peakDb.toFixed(1) + ' dBFS · rende
red in ' + seconds + (seconds === 1 ? ' second' : ' seconds')
        + (peak * gain > 1 ? ' · The mix is louder than 0 dBFS and may clip (turn on Normalize or lower
the master).' : '');
    } else {
      document.getElementById('ex-done-note').textContent = 'No sound was rendered. Check that the track
s are not muted and that clips are inside the range.';
    }
    showView('done');
```

```
  } catch (err) {
    showView('form');
    refreshForm();
    if (!(err && err.message === 'cancelled')) {
      const msg = document.getElementById('ex-msg');
      msg.textContent = 'Export failed: ' + (err && err.message ? err.message : 'unknown error');
      msg.hidden = false;
      if (window.console) console.warn('[export] 렌더링에 실패했습니다', err);
    }
  } finally {
    rendering = false;
  }
}
```

```
function stopExport() {
  if (!rendering) return;
  cancelFlag = true;
  document.getElementById('ex-stop').disabled = true;
  document.getElementById('ex-prog-title').textContent = 'Stopping…';
}
```

```
function downloadResult() {
  if (!result) return;
  const blob = new Blob([result.bytes], { type: 'audio/wav' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = result.fileName + '.wav';
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(function () { URL.revokeObjectURL(url); }, 60000);
}
```

```
export function closeExport() {
  const dlg = document.getElementById('export-modal');
  if (rendering) stopExport();
  if (dlg && dlg.open) dlg.close();
  result = null;
}
```

3.6 연결

```
export function initExport() {
  const dlg = document.getElementById('export-modal');
  if (!dlg) return;
  registerCommand('file.export', openExport, { label: 'Export audio…', group: 'file', keys: 'Ctrl+Shift+
R' });

  dlg.addEventListener('click', function (e) {
    const rate = e.target.closest('#ex-rate button[data-rate]');
    const depth = e.target.closest('#ex-depth button[data-depth]');
    if (rate) { pickSegment('ex-rate', 'rate', rate.dataset.rate); refreshForm(); }
    if (depth) { pickSegment('ex-depth', 'depth', depth.dataset.depth); refreshForm(); }
```

```
  });
  dlg.addEventListener('input', refreshForm);
  dlg.addEventListener('change', refreshForm);
  document.getElementById('ex-tail-auto').addEventListener('click', function () {
    if (!snap) return;
    document.getElementById('ex-tail').value = String(computeAutoTail());
    refreshForm();
  });
  document.getElementById('ex-go').addEventListener('click', startExport);
  document.getElementById('ex-stop').addEventListener('click', stopExport);
  document.getElementById('ex-download').addEventListener('click', downloadResult);
  document.getElementById('ex-cancel').addEventListener('click', closeExport);
  document.getElementById('ex-close').addEventListener('click', closeExport);
  document.getElementById('ex-x').addEventListener('click', closeExport);
  dlg.addEventListener('cancel', function (e) {
    if (rendering) { e.preventDefault(); stopExport(); }
  });
}
```

4. 화면의 구성

영역	내용
Format	WAV만 선택할 수 있습니다. MP3와 FLAC은 꺼 두고 "Phase 3에서 제공"이라고 알립니다.
Range	전체, 루프 구간, 선택한 클립, 직접 지정. 쓸 수 없는 항목은 흐리게 하고 이유를 오른쪽에 씁니다("No loop region", "Select clips on the timeline first").
Sample rate	44.1kHz, 48kHz. 처음에는 프로젝트의 샘플레이트를 따릅니다.
Bit depth	16bit, 24bit, 32bit float. 선택에 따라 아래 안내 문구가 바뀝니다.
MP3 bitrate	128, 192, 320을 꺼 둡니다. MP3 인코딩은 Phase 3에서 켭니다.
Normalize	켜면 목표 피크(기본 -0.3dBFS)를 입력할 수 있습니다.
Tail	초 단위 입력(0에서 30). Auto 버튼은 이펙트와 악기가 남기는 길이를 계산해 채웁니다.
File name	`{프로젝트명}_{YYYYMMDD_HHmm}`. 금지 글자는 `_`로 바뀌고, `.wav`를 직접 써도 중복되지 않습니다.
요약 줄	길이, 형식, 예상 크기. 값이 바뀔 때마다 다시 계산합니다.

5. 최신 문법으로 쓰면

```
// loadPrefs 와 clampNumber 를 최신 문법으로 쓴 모양
const loadPrefs = () => {
  try { return JSON.parse(localStorage.getItem(PREFS_KEY)) ?? {}; } catch { return {}; }
};
const clampNumber = (value, min, max, fallback) =>
  Number.isFinite(Number(value)) ? Math.min(max, Math.max(min, Number(value))) : fallback;

// 클립 선택 표시와 필터
const selectedClips = () => allClips().filter((c) => c.selected);
const pickSegment = (groupId, attr, value) =>
  document.getElementById(groupId).querySelectorAll('button').forEach((b) => { /* ... */ });
```

전통 문법 (실제 코드)	최신 문법
`raw ? JSON.parse(raw) : {}`	`JSON.parse(raw) ?? {}`
`catch (e) { ... }`	변수 없는 `catch { ... }`
`Array.prototype.forEach.call(목록, ...)`	`목록.forEach(...)` 또는 `[...목록].forEach`
`isFinite(n)`	`Number.isFinite(n)`

6. 왜 이렇게

선택	이유
창을 열 때 화면의 스냅샷을 찍습니다	창은 모달이라 열려 있는 동안 편집할 수 없습니다. 한 번 찍은 값을 계속 쓰면 값이 중간에 바뀌어 요약 줄과 결과가 어긋나는 일이 없습니다.
범위는 늘 "전체"로 시작합니다	클립이 선택된 채로 창을 열면 일부만 내보내기가 되기 쉽습니다. 모르고 곡의 한 조각만 받는 일을 막으려고, 선택한 클립은 사용자가 직접 고르게 했습니다.
"선택한 클립"은 타임라인에서 선택한 클립입니다	FL Studio의 Pattern 모드와 Ableton의 선택한 트랙처럼 "지금 고른 것만 내보내기"를 지원하려는 것입니다. 선택하지 않은 클립은 소리를 끄고, 트랙 이펙트와 믹서는 그대로 적용합니다.
화면의 클립 요소에서 선택 여부를 읽습니다	선택은 화면에만 있는 상태(`is-selected`)입니다. 재생에 넘기는 값을 모으는 함수(`collectAudioClips`, `collectMidiClips`)에 `selected` 한 줄만 더해, 같은 스냅샷을 재생과 내보내기가 함께 씁니다.
쓸 수 없는 범위를 숨기지 않고 흐리게 둡니다	사용자가 그런 범위가 있다는 것을 알고, 어떻게 쓰는지 안내문으로 알 수 있습니다. 쓸 수 없는 상태에서 선택되어 있었다면 자동으로 "전체"로 돌립니다.
샘플레이트, 비트 깊이, 정규화는 기억합니다	사용자는 보통 늘 같은 설정으로 내보냅니다. 범위, 꼬리, 파일명은 곡마다 다르므로 기억하지 않고 새로 정합니다. 브라우저 저장소를 못 쓰는 환경에서는 기본값으로 시작합니다.
처음 샘플레이트는 프로젝트의 값을 따릅니다	프로젝트가 48kHz인데 기억이 없으면 44.1kHz로 내보내서 소리를 다시 맞추는 일이 없게 합니다.
진행 화면의 안내는 "남은 시간"을 어림해서 보여 줍니다	10%가 지난 뒤에만 계산합니다. 처음에는 샘플을 불러오는 시간이 섞여서 값이 크게 틀리기 때문입니다.
렌더링 중 Esc는 닫지 않고 중지로 받습니다	바로 닫으면 렌더링이 눈에 보이지 않는 곳에서 계속 돕니다. 먼저 멈추고 설정 화면으로 돌아가게 합니다.
완료되어도 자동으로 내려받지 않습니다	결과를 확인(파일명, 피크, 소리가 비었는지)한 뒤 사용자가 Download를 누르게 합니다. PRD(FR-307)도 완료 시 다운로드 버튼을 둡니다.
소리가 비었으면 완료 화면이 알려 줍니다	뮤트했거나 범위에 클립이 없으면 무음 파일이 나옵니다. 그냥 두면 받은 파일을 열어 본 뒤에야 알게 됩니다.
피크가 0dBFS를 넘으면 경고를 붙입니다	정수 WAV는 그 값에서 잘려 소리가 거칠어집니다. 정규화를 켜거나 마스터를 낮추라고 안내합니다.
다운로드는 `Blob`과 임시 주소로 합니다	서버를 거치지 않아도 되고(클라이언트 렌더링, FR-302), 파일이 서버에 남지 않습니다. 임시 주소는 1분 뒤에 풉니다.
Ctrl+Shift+R을 가로챕니다	PRD와 메뉴의 단축키 표시가 Ctrl+Shift+R입니다. 시퀀서 화면은 작업 화면이어서 강력 새로고침보다 내보내기가 더 자주 쓰입니다. 입력칸에 커서가 있거나 다른 창이 열려 있으면 무시합니다.

7. 시험

- `docs/handoff/s1-11b_tests/ui_export.mjs` (46건)는 실제 시퀀서 화면(로그인한 시험 계정)에서 확인합니다. 시험용 트랙에 8초 사인파 클립 두 개(1에서 2마디, 5에서 6마디)를 놓고 시작하며, 내려받기를 가로채서 바이트를 페이지 안에서 `decodeAudioData`로 다시 읽습니다.

    - 창 열기: ↓ 버튼, FILE 메뉴(실제 마우스 클릭), Ctrl+Shift+R이 창을 엽니다. ✕, Cancel, Close가 닫습니다.
    - 기본 화면: 범위는 전체(`bar 1 – 6`), 루프와 선택 항목은 꺼져 있고, 파일명은 `{프로젝트명}_{YYYYMMDD_HHmm}`, 형식은 16bit이며 샘플레이트는 프로젝트를 따릅니다.
    - 기본값 내보내기: 파일이 RIFF, PCM, 스테레오, 16bit이고, 길이가 6마디이며, 1에서 2마디와 5에서 6마디에 소리(피크 0.4)가 있고 3에서 4마디는 무음입니다.
    - 옵션: 48kHz, 24bit, 정규화, 꼬리 1초가 파일에 반영됩니다. 피크가 0.966(-0.3dBFS)이고, 길이가 6마디 더하기 1초이며, 파일명의 `/`가 `_`로 바뀝니다.
    - 범위: 선택한 클립(길이 2마디), 루프 구간(길이 2마디), 직접 지정 3에서 4마디(빈 구간이라 길이 2마디에 완전한 무음, 완료 화면이 "No sound was rendered"라고 알림)가 맞습니다.
    - 잘못된 범위(끝이 시작보다 앞)는 내보내기 버튼이 꺼지고 안내가 뜹니다.
    - 중지: 진행 화면에서 Stop을 누르면 설정 화면으로 돌아오고 오류 안내가 없으며 파일이 만들어지지 않습니다. 렌더링 중 Esc는 창을 닫지 않고 "Stopping…"으로 바뀝니다.
    - 지난번에 고른 샘플레이트, 비트 깊이, 정규화를 기억합니다. 화면에서 JS 오류가 나지 않습니다.
- 시험 중 발견한 일 (모두 시험 쪽 문제였고 제품 코드는 그대로입니다):
    - 메뉴 항목은 프로그램이 만든 클릭(`isTrusted`가 아닌 클릭)을 무시합니다. 공용 명령 레지스트리가 같은 명령을 다시 부르는 무한 반복을 막으려는 기존 설계입니다. 시험이 실제 마우스 이벤트(`Input.dispatchMouseEvent`)로 메뉴를 누르도록 바꿨습니다.
    - 3번 단계에서 켠 정규화를 창이 기억해서, 다음 단계들의 피크가 0.4가 아니라 0.966으로 나왔습니다. 의도한 동작이므로 시험의 기대값을 바꾸고, 기억 기능의 확인을 겸하게 했습니다.
    - 시험 안의 여러 `const r` 선언이 한 번의 평가에서 충돌했습니다. 중괄호 블록으로 감쌌습니다.